



# **Parallel & Distributed Computing Lecture Week5: MPP and Dependency Analysis**

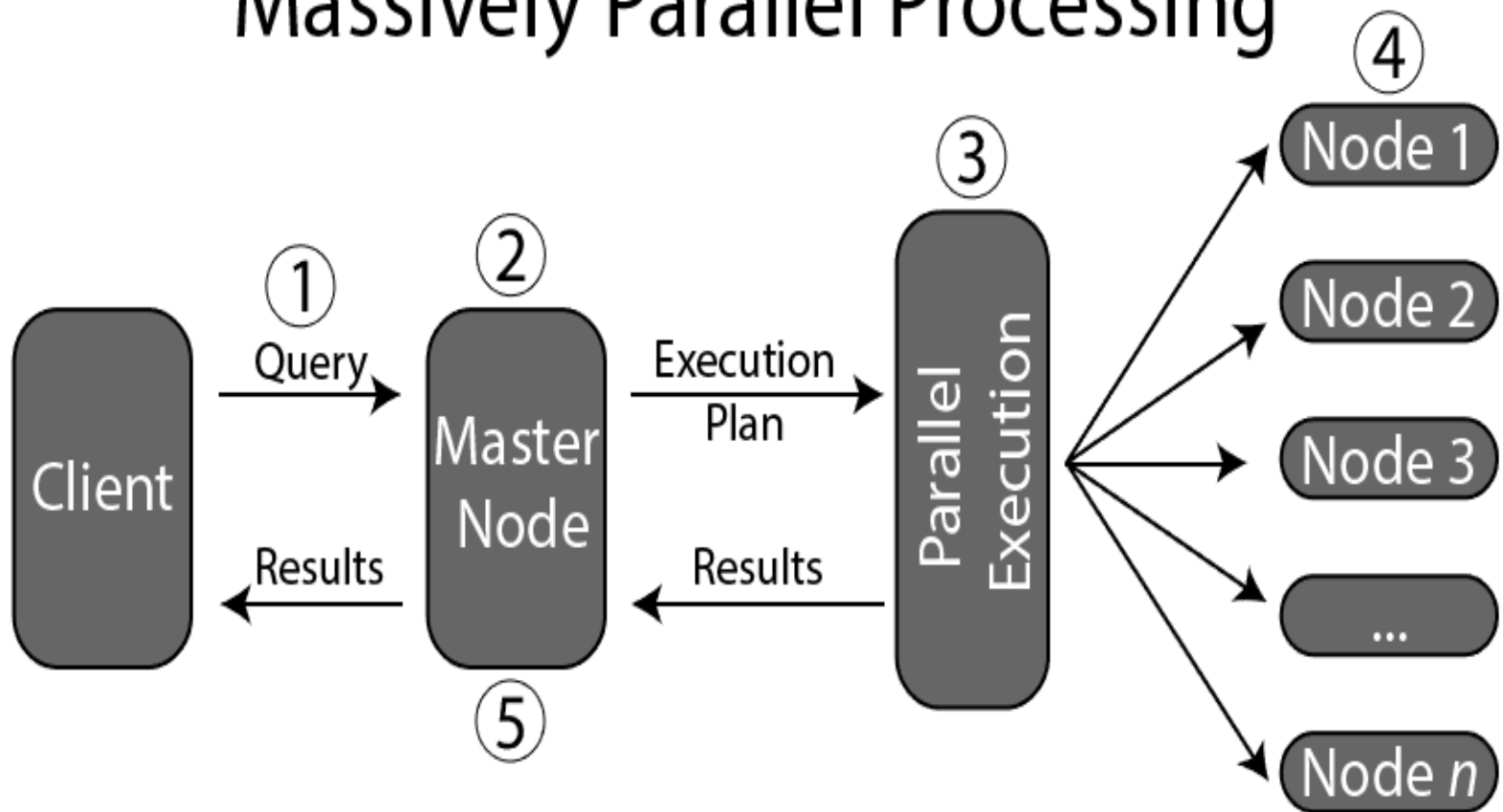
**Farhad M. Riaz**

[Farhad.Muhammad@numl.edu.pk](mailto:Farhad.Muhammad@numl.edu.pk)

**Department of Computer Science  
NUML, Islamabad**

# MPP Architecture

## Massively Parallel Processing



# **MPP Architecture and Dependence Analysis**

- Approaches
  - Parallel Algorithms
  - Parallel Language
  - Message passing (low-level)
  - Parallelizing compilers

# Parallelizing Compilers

- Automatically transform a sequential program into a parallel program.
  - Identify loops whose iterations can be executed in parallel.
  - Often done in stages.
- Q: Which loops can be run in parallel?
- Q: How should we distribute the work/data?

# Data Dependences

- **Flow dependence** - RAW. Read-After-Write. A "true" dependence. Read a value after it has been written into a variable.
- **Anti-dependence** - WAR. Write-After-Read. Write a new value into a variable after the old value has been read.
- **Output dependence** - WAW. Write-After-Write. Write a new value into a variable and then later on write another value into the same variable

# Example

Flow Dependency is a type of data dependency which occurs when an instruction depends on the result of a previous instruction. A Flow dependency is also known as read-after-write (RAW) or data dependency or true dependency

- $A = 90;$
- $B = A;$
- $C = A + D;$
- $A = 5;$

- i1: load r1, a;
- i2: add r2, r1, r1;
- instruction i2 uses r1 as a source. As a result, i2 cannot be directly implemented just before r1 has been loaded by i1. Thus, i2 is RAW-dependent on it. RAW dependencies are also known as flow dependencies. They are true dependencies because they cannot be abandoned.
- RAW dependencies can be divided into load-use and define-use dependencies
- i1: mul r1, r4, r5;
- i2: add r2, r1, r1;

# Example

- An anti-dependency is a type of data dependency which occurs when an instruction requires a value that is updated later. Anti-Dependency is also known as write-after-read (WAR). In the following example: Instruction 2 anti-depends on instruction 3 — the ordering of these instructions cannot be changed, nor can they be executed in parallel (possibly changing the instruction ordering), as this would affect the final value of A.
- $B = 3$
- $A = B + 1$
- $B = 7$



- i1: mul r1, r2, r3;
- i2: add r2, r4, r5;

# Examples

- An Output Dependency is a type of data dependency which occurs when the ordering of instructions affects the final output value of any variable. Output Dependency is also known as write-after-write (WAW). In the example below, there is an output dependency between instructions 3 and 1 — changing the order of those instructions in this example will change the final value of A, thus these instructions cannot be executed in parallel.
- $B = 3$
- $A = B + 1$
- $B = 7$

- i1: mul r1, r2, r3;
- i2: add r1, r4, r5;

# Dependencies

- A parallelizing compiler must identify loops that do not have dependences BETWEEN ITERATIONS of the loop.
- Example:

```
do I = 1, 1000
    A(I) = B(I) +
    C(I)  D(I) = A(I)
end do
```

# Example

Fork one thread for each processor

Each thread executes the loop:

```
do I = my_lo, my_hi
```

```
    A(I) = B(I) + C(I)
```

```
    D(I) = A(I)
```

```
end do
```

Wait for all threads to finish before proceeding.

# Another Example

```
do I = 1, 1000
```

```
    A(I) = B(I) + C(I)    D(I) = A(I+1)
```

```
end do
```

# Yet Another Example

```
do I = 1, 1000  
    A( X(I) ) = B(I) + C(I)  
    D(I) = A( X(I) )  
end do
```

*That's all for today!!*